Advanced Internet of Things Technology

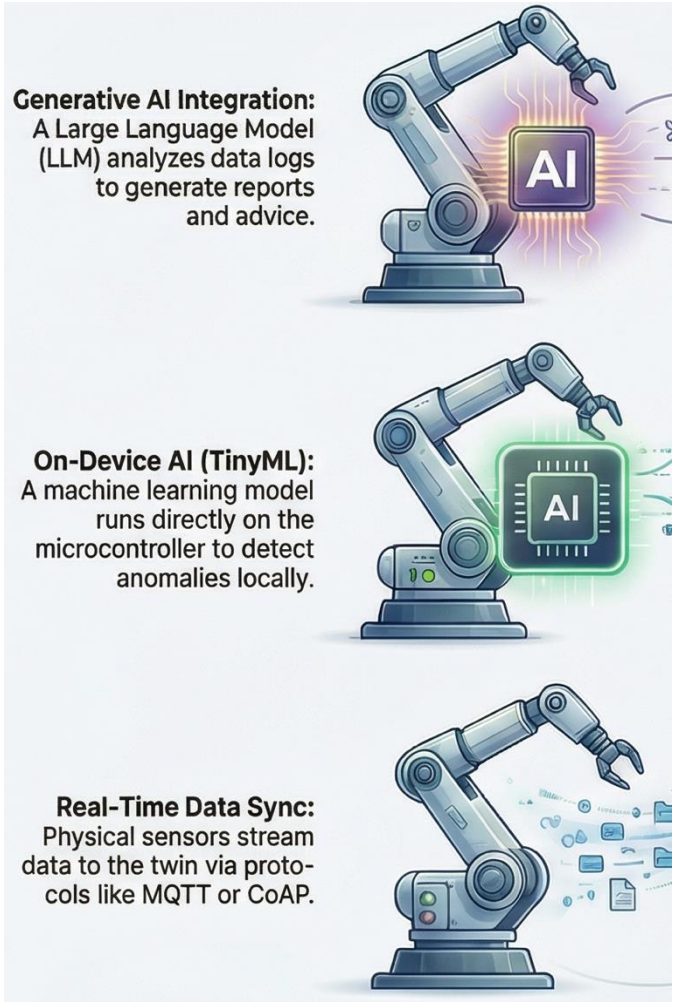
Week 2: Edge Intelligence I - Tiny ML for IoT Devices

TS. Huỳnh Văn Đặng
Department of Computer Networks
Faculty of Computer Networks and Communications
University of Information Technology, VNUHCM
Email: danghv@uit.edu.vn

December 2025

Week 2: The TinyML for IoT

VIET NAM NATIONAL UNIVERSITY HCMC
UNIVERSITY OF INFORMATION TECHNOLOGY
VNUHCM - UIT



Course context

- This week shifts the focus from conceptual Digital Twin foundations to practical edge intelligence.
- Emphasis is placed on how machine learning can run on constrained edge devices.
- The material bridges cloud-centric IoT designs and edge-first autonomous systems.

Why this topic matters

- Many IoT systems fail due to latency, bandwidth, or privacy constraints.
- TinyML enables local intelligence under strict hardware limits.
- System-level thinking is required to make ML deployable on real devices.

What we will learn this week

- The motivations for pushing intelligence to the edge.
- The complete TinyML workflow from data to deployment.
- How constraints shape model and system design decisions.

Part A – Why move intelligence to the edge?

The cloud-centric bottleneck

- Cloud inference introduces unpredictable round-trip latency.
- Raw data streaming consumes excessive bandwidth.
- Privacy risks increase when sensitive data leaves devices.
- Connectivity loss breaks cloud-dependent autonomy.

Latency as a primary driver

- Control and safety loops require millisecond responses.
- Network delays dominate end-to-end latency.
- Local inference ensures predictable timing behaviour.

Bandwidth efficiency through abstraction

- Sensors generate high-rate raw data streams.
- Edge processing extracts events and features.
- Only distilled insights are transmitted upstream.

Energy constraints

- Wireless transmission is energy intensive.
- Local inference reduces radio usage.
- Optimised models keep computation energy affordable.

Why move intelligence to the edge? (cont.)

Operational resilience

- Real-world networks experience outages and instability.
- Edge intelligence enables offline-capable operation.
- Cloud services become supportive rather than critical.

Privacy and security motivations

- Sensitive data remains local on the device.
- Reduced transmission lowers attack surfaces.
- Compliance requirements favour data minimisation.

Edge intelligence as a system decision

- No single driver applies to all applications.
- Trade-offs differ across domains and deployments.
- Edge and cloud play complementary roles.

The Motivation: Why IoT Demands Intelligence at the Edge

Relying solely on the cloud for IoT introduces critical bottlenecks in latency, cost, and resilience. Edge intelligence moves computation closer to data sources to solve these fundamental challenges.

PERFORMANCE

- **Ultralow Latency:** Eliminates the cloud round-trip delay, enabling immediate responses for mission-critical systems where milliseconds matter.
- **Bandwidth Efficiency:** Pre-processes data locally, sending only relevant results upstream. This *"alleviates the burden on the core network"* and preserves bandwidth.

RESILIENCE

- **Operational Uptime:** Reduces dependence on constant connectivity, allowing devices to operate autonomously if the network or cloud service fails.
- **Data Privacy & Security:** Processes sensitive information like video or health data locally, minimizing exposure of raw data to external servers.

EFFICIENCY

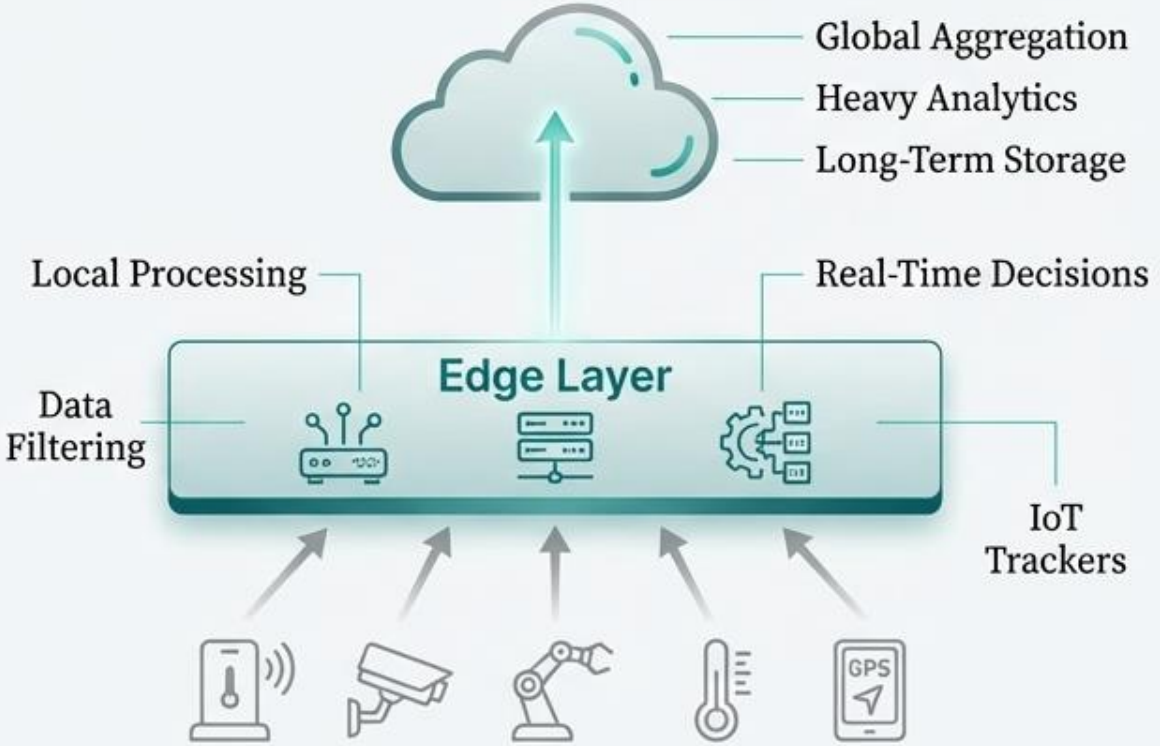
- **Cost & Scalability:** Reduces cloud computing and data egress costs, preventing bottlenecks and enabling scalable deployments with inexpensive hardware.
- **Energy & Battery Constraints:** Allows devices to remain in low-power states by transmitting only distilled insights, which is more power-effective than running radios at high duty cycles.

The New Architecture: Placing an Intelligent Layer Between Devices and the Cloud

Traditional Cloud-Centric IoT

High Latency, High Bandwidth Usage, Single Point of Failure

Edge-Enabled Architecture



In a three-tier architecture, edge nodes or gateways act as a bridge, executing time-critical tasks near the data source and interacting with the cloud only when necessary.

Checkpoint 1 – Discussion

VIET NAM NATIONAL UNIVERSITY HCMC
UNIVERSITY OF INFORMATION TECHNOLOGY
VNUHCM - UIT

Application analysis

Choose an IoT application you are familiar with.

Identify the top two dominant constraints.

Explain why these constraints dominate.

Edge–cloud split

Decide what must run on-device.

Decide what can be sent to the cloud.

Justify your choices using system arguments.

Reflection

Compare decisions across groups.

Observe differences between application domains.

Prepare to map choices onto the TinyML workflow.

Part B – TinyML end-to-end workflow

VIET NAM NATIONAL UNIVERSITY HCMC
UNIVERSITY OF INFORMATION TECHNOLOGY
VNUHCM - UIT

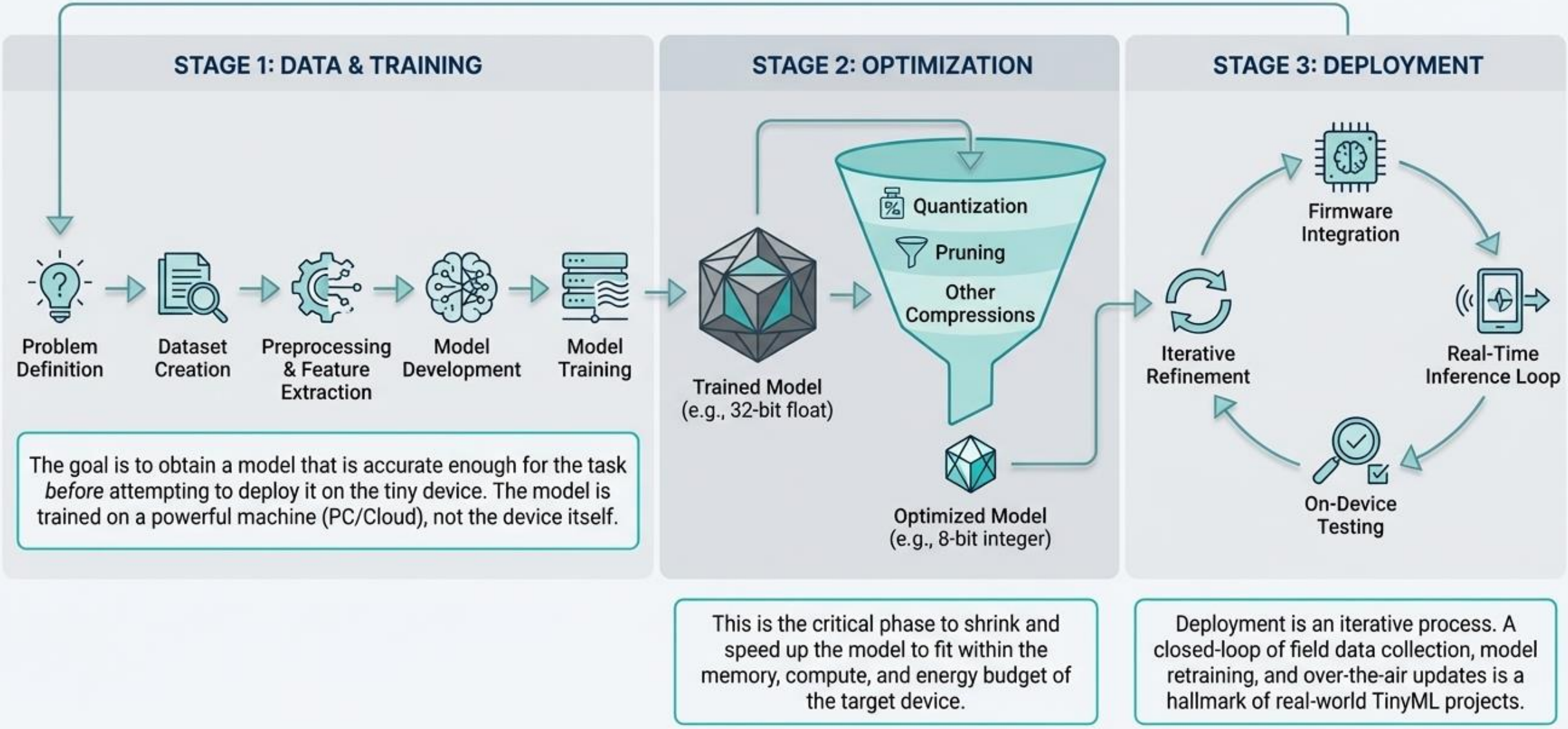
Workflow overview

- Problem definition and data collection.
- Preprocessing and feature extraction.
- Model development and training.
- Optimisation for deployment.
- Firmware integration and inference.

Design principle

- Deployment constraints influence all stages.
- Optimisation is not optional in TinyML.
- Iteration is expected throughout the pipeline.

The TinyML Workflow: From Data Collection to On-Device Inference



Step 1 – Problem definition

Task specification

- Define classification, detection, or regression objectives.
- Determine inference frequency and trigger conditions.
- Specify acceptable error characteristics.

Constraint specification

- Set latency, memory, and energy budgets.
- Identify environmental variability.
- Define required system actions.

Evaluation criteria

- Accuracy and robustness metrics.
- Inference time and memory usage.
- Power consumption under operation.

Step 1 – Data collection

Representativeness

- Capture data under real operating conditions.
- Include noise and edge cases.
- Avoid overly clean laboratory datasets.

Labelling strategy

- Define consistent annotation rules.
- Address class imbalance early.
- Validate labels against deployment behaviour.

Dataset management

- Use proper train/validation/test splits.
- Preserve temporal structure in time-series.
- Maintain a realistic test set.

Step 2 – Preprocessing

Signal preparation

- Denoise and normalise raw signals.
- Align sampling rates across sensors.
- Remove irrelevant artefacts.

Windowing for streaming data

- Use fixed-size sliding windows.
- Maintain circular buffers on-device.
- Balance window size and responsiveness.

Deployment consistency

- Preprocessing must match training exactly.
- Prefer deterministic and integer-friendly operations.
- Avoid computationally heavy transformations.

Step 2 – Feature extraction

Purpose of feature extraction

- Reduce input dimensionality.
- Improve robustness under noise.
- Lower model complexity requirements.

Feature examples

- Frequency-domain features for signals.
- Statistical summaries for sensor windows.
- Domain-informed features aligned with physics.

Trade-offs

- Features simplify models but reduce flexibility.
- End-to-end learning requires more capacity.
- Choice depends on data and constraints.

Step 3 – Model development

Architecture selection

- Prefer small, efficient model families.
- Avoid layers with large intermediate activations.
- Treat peak SRAM usage as a key metric.

Multi-objective thinking

- Accuracy alone is insufficient.
- Memory and latency constraints dominate.
- The best model is the deployable model.

Operator awareness

- Runtime support limits usable operations.
- Simplify or replace unsupported layers.
- Plan for deployment early.

Step 3 – Training

Training environment

- Training occurs off-device.
- Use standard ML frameworks.
- Maintain reproducibility.

Validation strategy

- Track accuracy and confusion metrics.
- Estimate memory and latency early.
- Detect overfitting and data leakage.

Iterative refinement

- Adjust data, features, or architecture.
- Rebalance accuracy and efficiency.
- Stop when constraints are met.

Step 4 – Optimisation

Why optimisation is central

- Raw models rarely fit on MCUs.
- Optimisation enables deployment feasibility.
- Memory, latency, and power are all targeted.

Core techniques

- Quantisation reduces numeric precision.
- Pruning removes redundant parameters.
- Techniques are often combined.

Evaluation discipline

- Measure accuracy after each step.
- Test memory and latency on target hardware.
- Keep a clear optimisation log.

Making it Tiny: The Core Techniques of Model Optimization

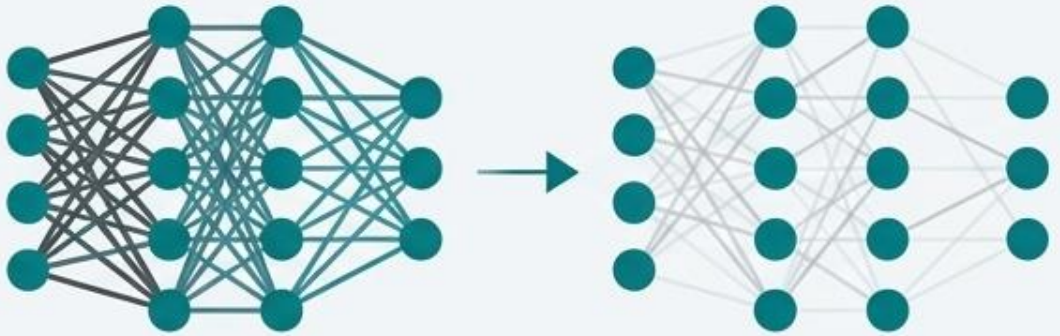
Quantization: Reducing Numeric Precision

The process of converting a model's parameters and computations from high-precision numbers (like 32-bit floating-point) to low-precision numbers (like 8-bit integers).

- 4x reduction in memory usage.
- Faster inference due to simpler integer arithmetic.
- Minor accuracy trade-off for major efficiency gains.

Pruning: Removing Unnecessary Connections

Systematically removing redundant or near-zero weights from a neural network to create a smaller, sparser model.



- Dramatically reduces parameter count and computations.
- Often combined with quantization to shrink models by an order of magnitude.
- Lost accuracy can be recovered by fine-tuning the pruned model.

Real-time inference loop

Execution pattern

- Sense, preprocess, infer, act.
- Maintain real-time constraints.
- Support continuous or event-driven modes.

Timing constraints

- Control applications need low latency.
- Monitoring allows slower inference.
- Match loop timing to application needs.

Memory safety

- No virtual memory on MCUs.
- Avoid dynamic allocation.
- Validate behaviour under quantisation.

Case study: MCUNET

VIET NAM NATIONAL UNIVERSITY HCMC
UNIVERSITY OF INFORMATION TECHNOLOGY
VNUHCM - UIT

The Resource Chasm: Cloud vs. Mobile vs. Tiny AI		
Platform	**Memory (SRAM)**	**Storage (Flash)**
NVIDIA V100	16 GB	TB~PB
iPhone 11	4 GB	>64 GB
STM32F746 MCU	320 kB	1 MB

~13,000x Less Memory

>64,000x Less Storage

- Project Page: http://tinyml.mit.edu
- Paper: *MCUNet: Tiny Deep Learning on IoT Devices*
- Code: https://github.com/mit-han-lab/mcunet

MCUNet: Tiny Deep Learning on IoT Devices

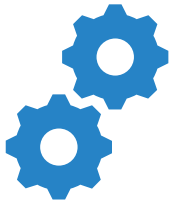
Cloud AI — ResNet

Mobile AI — MobileNet

Tiny AI — MCUNet

MCUNet overview

VIET NAM NATIONAL UNIVERSITY HCMC
UNIVERSITY OF INFORMATION TECHNOLOGY
VNUHCM - UIT

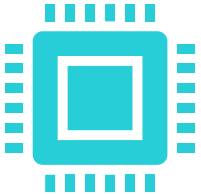


What is MCUNet?

An end-to-end TinyML deployment framework.

Combines neural architecture search and runtime optimisation.

Targets strict MCU-level constraints.



Key components

TinyNAS: constraint-aware neural architecture search.

TinyEngine: memory-efficient inference runtime.

Co-design loop expands feasible model space.

Key insight

Reducing runtime overhead enables better models.

Better models justify richer search spaces.

Co-design outperforms isolated optimisation.

MCUNET experimental results

VIET NAM NATIONAL UNIVERSITY HCMC
UNIVERSITY OF INFORMATION TECHNOLOGY
VNUHCM - UIT

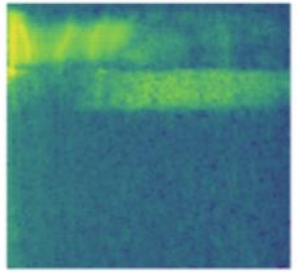
We focus on large-scale datasets to reflect real-life use cases.

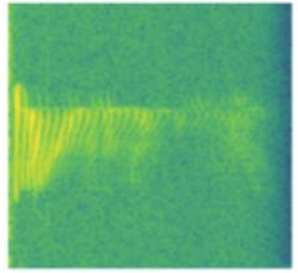
Datasets:
(1) ImageNet-1000
(2) Wake Words
- Visual: Visual Wake Words
- Audio: Google Speech Commands

yes no

(a) 'Person'

(b) 'Not-person'





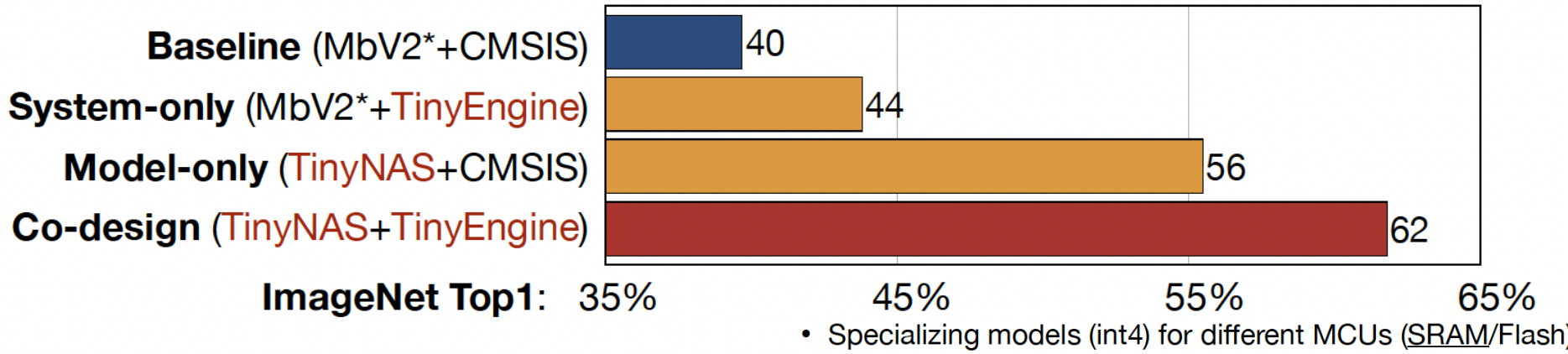
HANLAB

Source: https://hanlab18.mit.edu/projects/tinyml/mcunet/assets/MCUNet-slides.pdf

MCUNET experimental results

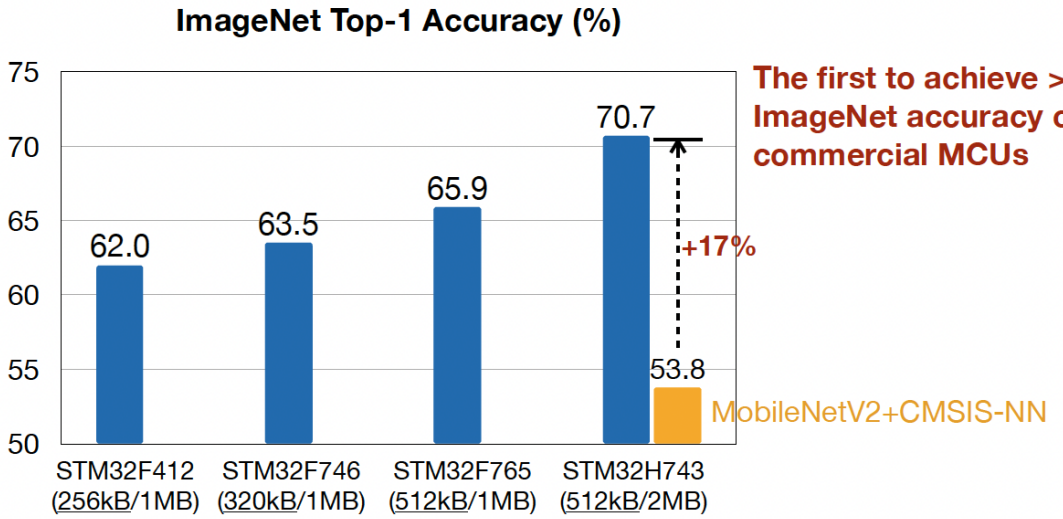
VIET NAM NATIONAL UNIVERSITY HCMC
UNIVERSITY OF INFORMATION TECHNOLOGY
VNUHCM - UIT

- ImageNet classification on STM32F746 MCU (**320kB SRAM**, **1MB Flash**)



- Specializing models (int4) for different MCUs (SRAM/Flash)

Source:
https://hanlab18.mit.edu/projects/tinyml/mcunet/assets/MCUNet-slides.pdf



The first to achieve >70% ImageNet accuracy on commercial MCUs

MCUNet performance insights

Memory efficiency gains

Significant reduction in peak SRAM usage.

Enables deeper or wider models.

Makes CNNs viable on small MCUs.

Accuracy improvements

Co-design allows better architectures.

Outperforms manually compressed baselines.

Maintains accuracy under tight constraints.

System-level takeaway

Runtime choices affect algorithm choices.

Co-design is essential for TinyML scalability.

Think beyond model-only optimisation.

The Hardware Spectrum: Platforms for On-Device Machine Learning

MICROCONTROLLERS (MCUs)

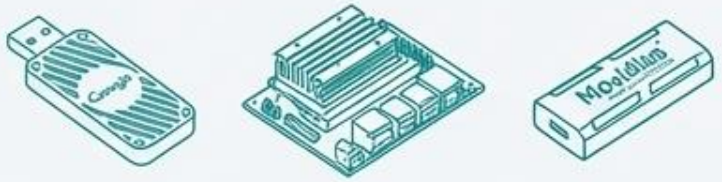
Characteristics: Ultra-low power (mW/µW), low cost, limited memory (tens/hundreds of KB RAM), limited compute (MHz), often no OS.

Best for: "Always-on" sensing, simple classification, battery-powered devices where energy is paramount.

Example: An Arduino Nano 33 BLE Sense with 256 KB RAM running real-time gesture recognition.

Hybrid chips (MCUs with integrated NPUs like Arm Ethos-U) are emerging to offer the best of both worlds.

EDGE AI ACCELERATORS (NPUs/SoCs)



Characteristics: Higher performance (billions of ops/sec), higher power (Watts), more memory (MBs/GBs), requires SDKs.

Best for: Real-time video analysis, multi-sensor fusion, complex models that are too large for an MCU.

Example: An Edge TPU executing a MobileNet vision model at dozens of frames per second.

Ideas for project!

- Wake-word detection or sound event recognition
- Vibration-based anomaly detection
- Low-resolution image classification on MCUs
- Edge extracts semantic features or events
- Digital twin receives abstracted state updates

Project Idea Set 1 – Sensing-Centric TinyML Applications

Always-on audio intelligence

- Wake-word detection or sound event recognition
- Continuous audio streaming with sliding windows
- Strong focus on low latency and low power consumption

Vibration-based anomaly detection

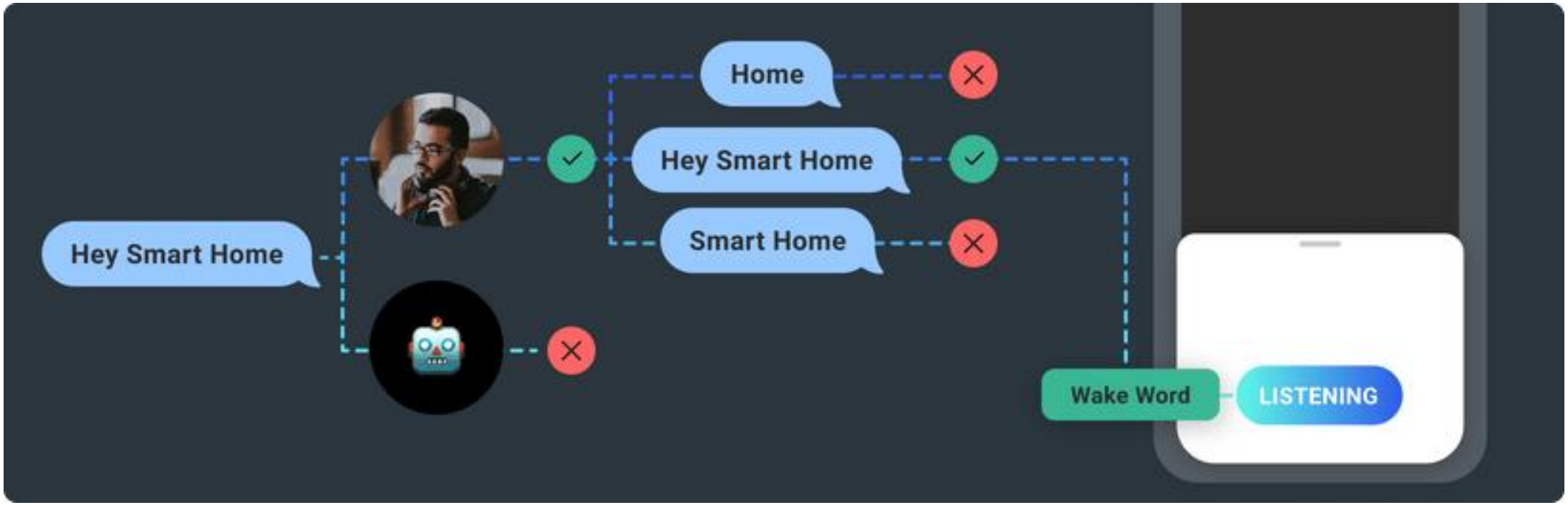
- Monitoring motors, pumps, or mechanical structures
- Time-series feature extraction and anomaly detection
- Event-driven reporting instead of raw data streaming

Key learning focus

- Streaming inference and window-based processing
- Accuracy versus false-alarm trade-offs
- Bandwidth and energy efficiency at the edge

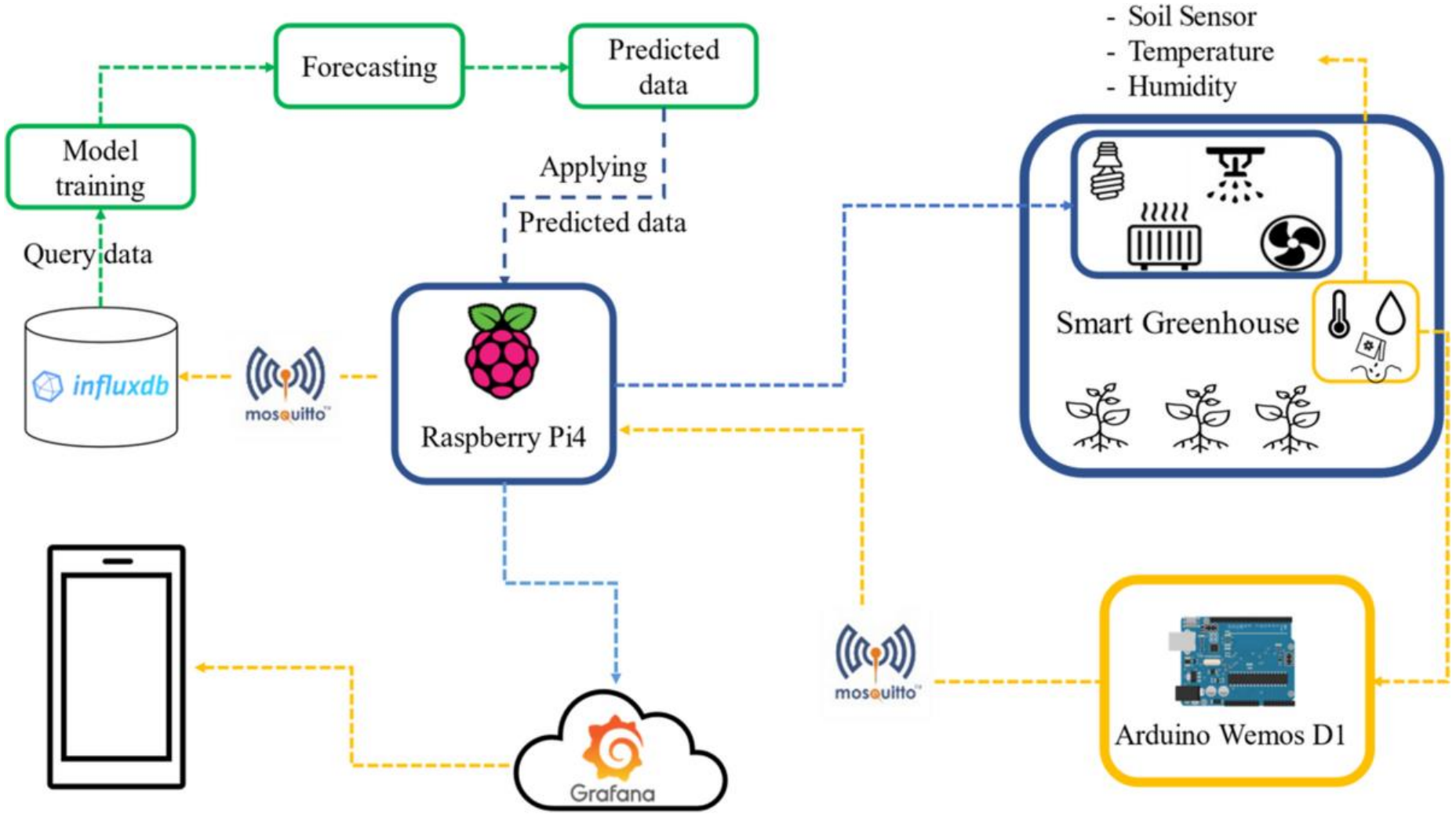
Wake-word detection or sound event recognition

https://www.spokestack.io/features/wake-word



Artificial Intelligent IoT-Based Cognitive Hardware for Agricultural Precision Analysis

VIET NAM NATIONAL UNIVERSITY HCMC
UNIVERSITY OF INFORMATION TECHNOLOGY
VNUHCM - UIT



https://doi.org/10.1007/s11036-023-02256-x

Project Idea Set 2 – Vision and System-Level Edge Intelligence

Tiny vision under memory constraints

- Low-resolution image classification on MCUs
- Emphasis on peak activation memory
- Inspiration from MCUNet-style co-design

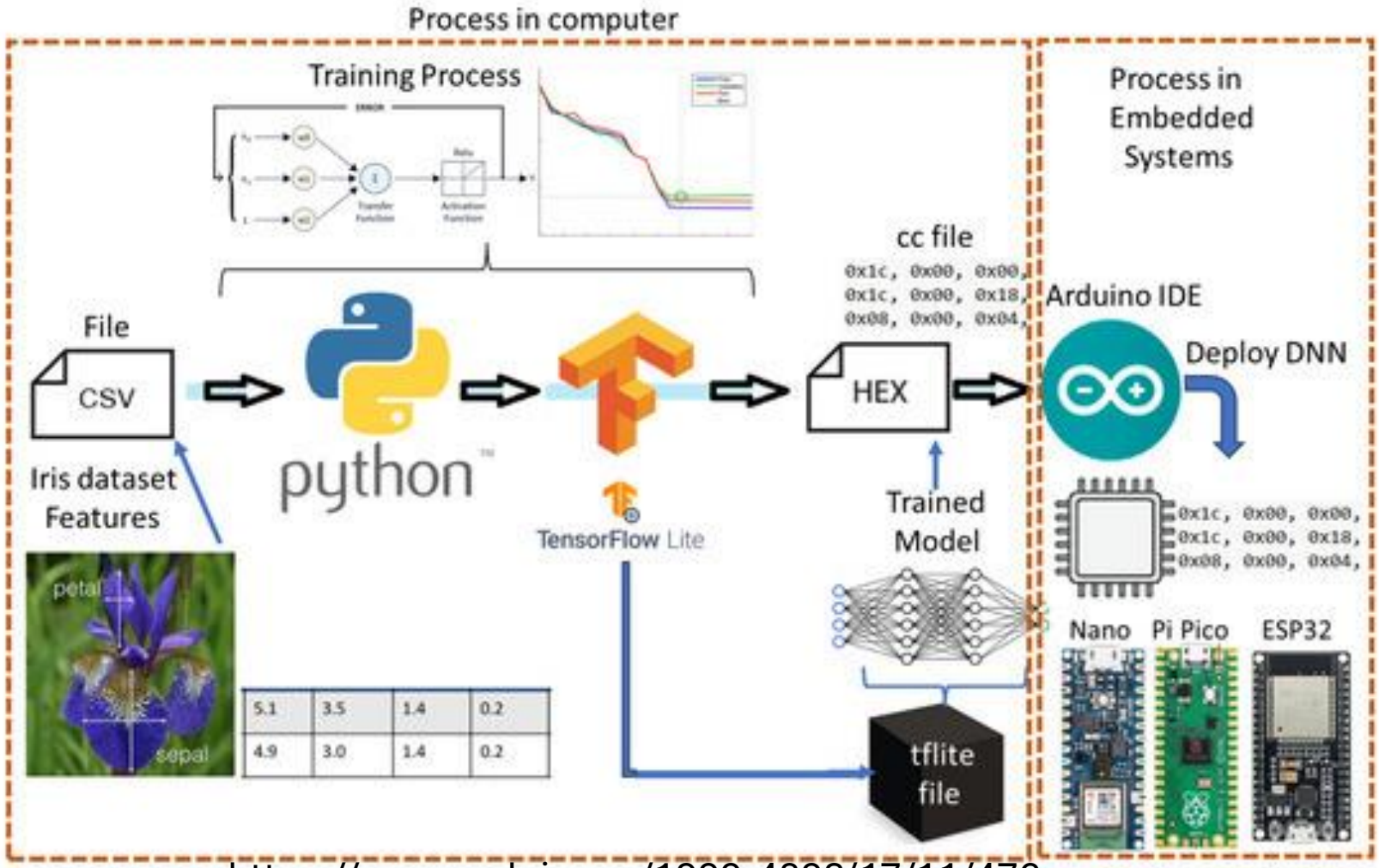
Edge-assisted digital twin update

- Edge extracts semantic features or events
- Digital twin receives abstracted state updates
- Reduced communication and improved scalability

Privacy-preserving edge filtering

- Edge decides what data can leave the device
- Prevents raw image, audio, or health data leakage
- Demonstrates privacy-by-design principles

Embedded Vision Systems-Based Tiny Machine Learning



https://www.mdpi.com/1999-4893/17/11/476

Part C – Edge LLMs and TinyChatEngine

Why edge LLMs?

- Latency-sensitive interaction.
- Privacy-preserving inference.
- Reduced cloud dependency.

Challenges of LLMs on edge

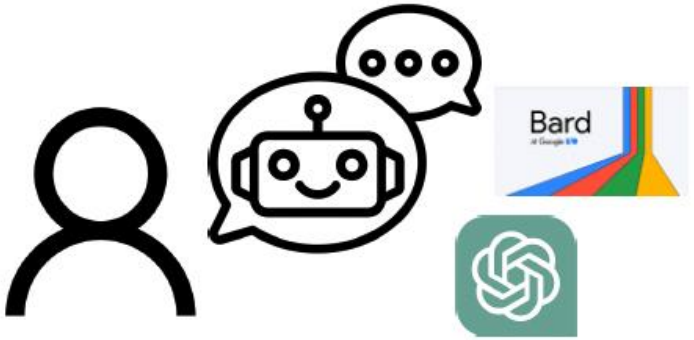
- Extremely large parameter counts.
- High memory and compute demands.
- Need aggressive compression.

TinyChatEngine overview

- Lightweight C/C++ inference engine.
- Toolchain for model conversion.
- Designed for constrained devices.

Deployment of Large Language Models

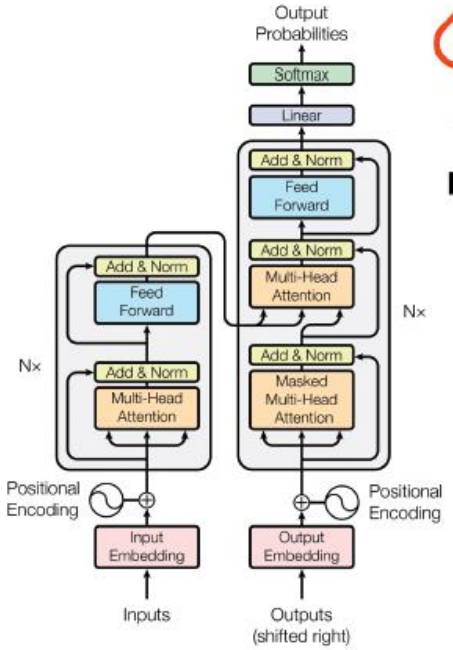
- Transformer-based large language models power impactful applications.



Online Chatbot

GitHub Copilot

Software Development



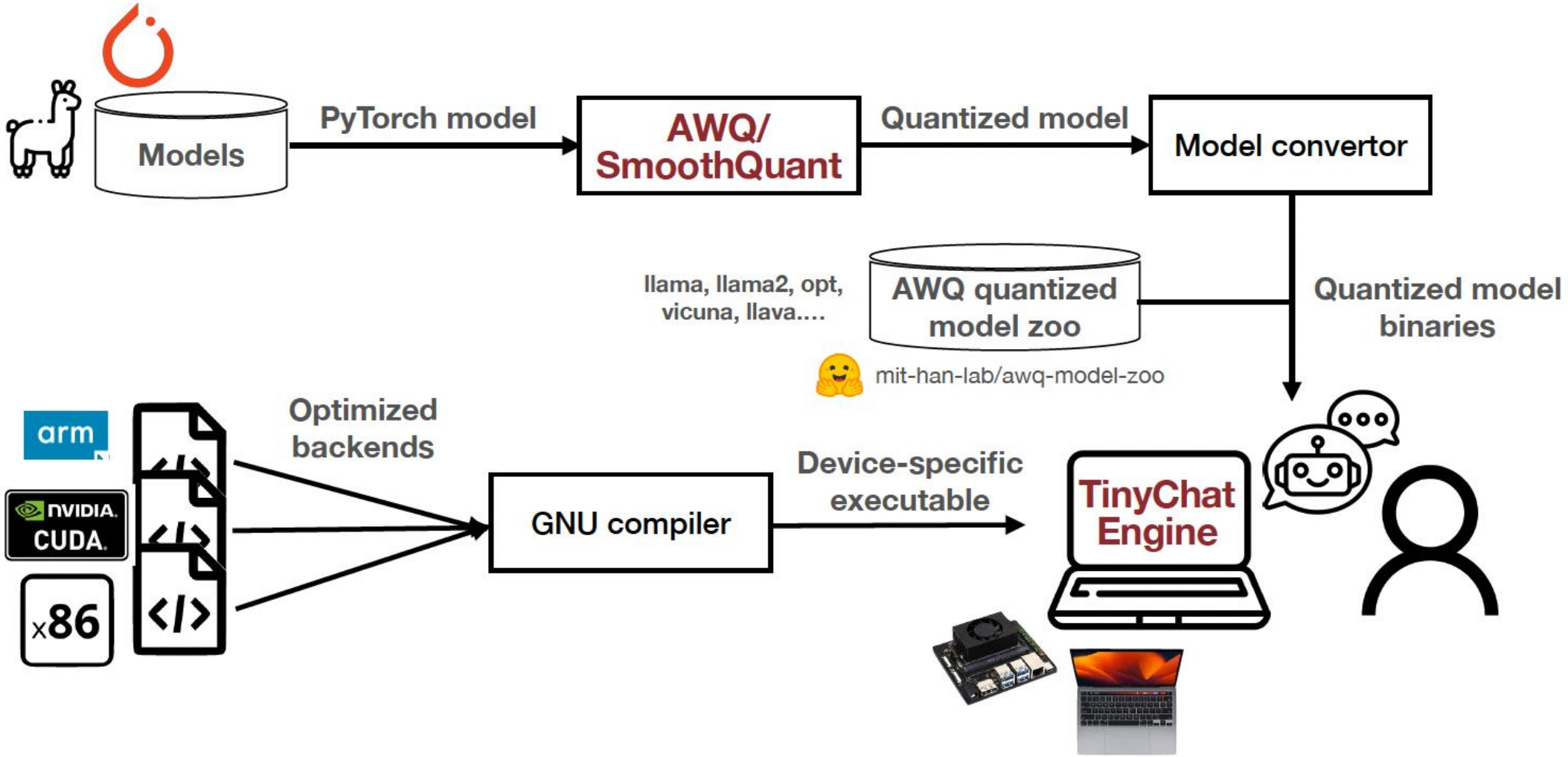
?

Cloud service

Edge devices

Source: Tiny ChatEngine Slides, Wei-Ming Chen

Workflow of Deploying a Model with TinyChatEngine



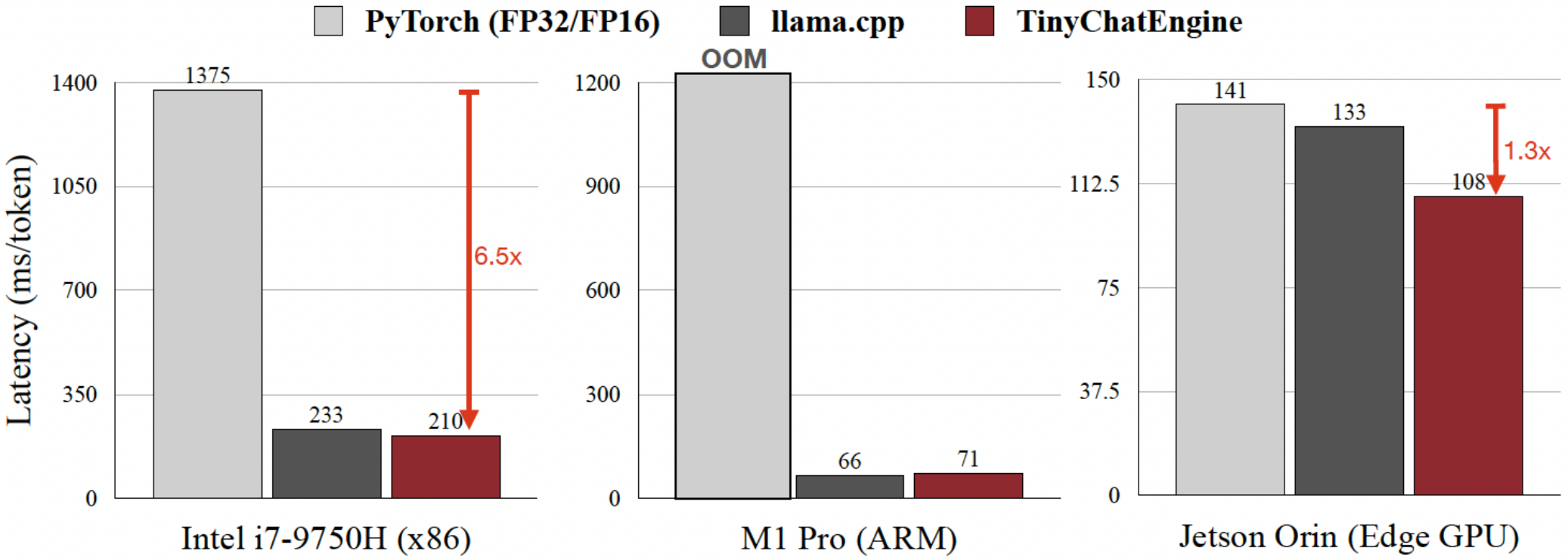
Source: Tiny ChatEngine Slides, Wei-Ming Chen

https://github.com/mit-han-lab/TinyChatEngine

Efficient LLM Deployment on Edge Devices

VIET NAM NATIONAL UNIVERSITY HCMC
UNIVERSITY OF INFORMATION TECHNOLOGY
VNUHCM - UIT

- TinyChatEngine achieves fast text generation for LLaMA2-7B on various devices



Source: Tiny ChatEngine Slides, Wei-Ming Chen

Discussion

VIET NAM NATIONAL UNIVERSITY HCMC
UNIVERSITY OF INFORMATION TECHNOLOGY
VNUHCM - UIT

Privacy reasoning

- What LLM functions must stay on-device?
- What can be safely summarised and sent?
- Consider regulatory and ethical factors.

System integration

- How would an edge LLM fit your project?
- Interaction with TinyML pipelines.
- Interaction with digital twins.

Reflection

- Edge LLMs as an extension of TinyML.
- Increasing importance in future systems.
- Prepare for advanced topics.

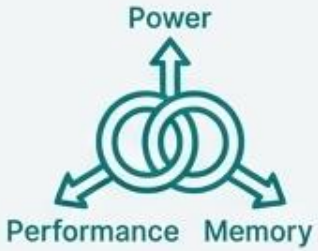
The Edge Inteligence Paradigm: A Synthesis

1. Edge is Essential for Real-Time IoT

It overcomes the fundamental latency, bandwidth, and resilience limitations of cloud-only architectures.

2. TinyML Makes AI Ubiquitous

Optimization techniques like quantization and pruning embed powerful analytics into energy-efficient microcontrollers at the network's edge.



3. Hardware is a Deliberate Trade-off

Choosing between MCUs and NPUs requires balancing the demands of the ML task with strict power, performance, and memory constraints.

4. Synergy with Digital Twins Enables Autonomous Control

The edge-twin combination creates proactive systems that not only monitor the physical world but can predict and prevent problems in a closed loop.

The trend is clear: pushing intelligence to the edge is necessary for creating IoT systems that are scalable, efficient, autonomous, and secure.

Wrap-up and project alignment

Key takeaways

- Edge intelligence is constraint-driven.
- TinyML workflow enables deployment.
- Co-design unlocks feasibility.

Project guidance

- Select application domain early.
- Identify dominant constraints.
- Align model and hardware choices.

Looking ahead

- Next week: distributed edge intelligence.
- Increasing autonomy and coordination.
- Preparation for project milestones.

References

[1] J Lin, WM Chen, Y Lin, C Gan, S Han, “Mcunet: Tiny deep learning on iot devices” in Proc. Advances in neural information processing systems, 2020.

[2] Ji Lin Ligeng Zhu Wei-Ming Chen Wei-Chen Wang Song Han, “*Tiny Machine Learning: Progress and Futures”*, arXiv:2206.15472

Useful tutorial: https://medium.com/@sucheta963/tinyml-running-deep-learning-models-on-microcontrollers-a1524a69e98c

Some graphics/slides are generated by NotebookLM with provided sources!